📋 Comandos de Configuración Inicial

Estos comandos los usas una sola vez para configurar tu identidad y entorno .

Comando	Descripción
`git config --global user.name "Tu Nombre"`	Establece tu nombre para todos los commits
`git config --global user.email "tu@email.com"`	Establece tu email para todos los commits
`git config --global core.editor "code --wait"`	Define VS Code como editor predeterminado
`git config --global init.defaultBranch main`	Cambia el nombre de la rama por defecto a "ma
`git config --list`	Muestra toda tu configuración actual

🗁 Crear y Clonar Repositorios

Comando	Descripción
`git init`	Inicia un nuevo repositorio Git en la carpeta actual
`git clone <url>`	Clona (descarga) un repositorio remoto a tu máquina
`git clone <url> <nombre-carpeta>`	Clona en una carpeta con nombre específico

📝 Comandos Básicos para Snapshots (Guardar Cambios)

Estos son los comandos que más usarás en tu día a día .

Comando	Descripción
`git status`	Muestra el estado de tus archivos (modificados, en staging
`git add <archivo>`	Agrega un archivo específico al área de staging
`git add .`	Agrega **todos** los archivos modificados al staging
`git add *.js`	Agrega todos los archivos con extensión .js
`git reset <archivo>`	Quita un archivo del área de staging (lo "des-stagia")
`git commit -m "mensaje"`	Crea un commit con los cambios del staging
`git commit -am "mensaje"`	Add + Commit en un solo paso (solo archivos ya rastreado
`git commit --amend -m "nuevo mensaje"`	Modifica el mensaje del último commit

🔍 Ver el Historial y Diferencias

Comando	Descripción
`git log`	Muestra todo el historial de commits
`git log --oneline`	Historial resumido (1 línea por commit)
`git log --graph --oneline --all`	Historial con gráfico de ramas
`git log -n 5`	Muestra solo los últimos 5 commits
`git log --author="Nombre"`	Filtra commits por autor

Comando	Descripción
`git log --since="2 weeks ago"`	Filtra commits desde hace 2 semanas
`git diff`	Muestra cambios no stagheados
`git diff --staged`	Muestra cambios que sí están en staging
`git diff HEAD`	Muestra diferencias entre directorio y último commit

🌿 Ramas (Branches)

Comando	Descripción
`git branch`	Lista todas las ramas locales
`git branch -a`	Lista ramas locales y remotas
`git branch <nombre>`	Crea una nueva rama
`git checkout <rama>`	Cambia a una rama existente
`git checkout -b <rama>`	Crea y cambia a una nueva rama
`git switch <rama>`	Cambia de rama (nuevo, más intuitivo)
`git switch -c <rama>`	Crea y cambia a nueva rama (nuevo)
`git branch -d <rama>`	Elimina una rama (solo si ya está fusionada)
`git branch -D <rama>`	Elimina una rama forzosamente
`git branch -m <nuevo-nombre>`	Renombra la rama actual

Comando	Descripción
`git merge <rama>`	Fusiona una rama con la rama actual
`git merge --abort`	Cancela una fusión con conflictos

🌐 Trabajar con Repositorios Remotos (GitHub, GitLab, etc.)

Comando	Descripción
`git remote -v`	Muestra los repositorios remotos configurados
`git remote add origin <url>`	Conecta tu repo local con uno remoto
`git remote set-url origin <nueva-url>`	Cambia la URL del remoto
`git fetch origin`	Descarga cambios del remoto **sin fusionarlos**
`git pull origin <rama>`	Descarga y fusiona cambios del remoto
`git push origin <rama>`	Sube tus commits al repositorio remoto
`git push -u origin <rama>`	Push + establece la rama upstream (solo la primera vez
`git push --all origin`	Sube **todas** las ramas
`git push origin --delete <rama>`	Elimina una rama en el remoto

⏎ Deshacer Cambios

Comando	Descripción
`git checkout -- <archivo>`	Descarta cambios en un archivo (¡cuidado!)
`git restore <archivo>`	Restaura un archivo a su último commit (nuevo)
`git restore --staged <archivo>`	Quita un archivo del staging (lo des-stagia)
`git reset HEAD~1`	Deshace el último commit pero mantiene los cambios
`git reset --soft HEAD~1`	Deshace el último commit, cambios quedan en staging
`git reset --hard HEAD~1`	Deshace el último commit **y borra los cambios** (¡peligroso!)
`git revert <commit-hash>`	Crea un nuevo commit que deshace los cambios de uno anterior (seguro)

🏷️ Etiquetas (Tags)

Comando	Descripción
`git tag`	Lista todas las etiquetas
`git tag -a v1.0 -m "mensaje"`	Crea una etiqueta anotada en el commit actual
`git tag -d <nombre>`	Elimina una etiqueta local
`git push origin <tag>`	Sube una etiqueta al remoto
`git push --tags`	Sube todas las etiquetas

🧹 Comandos Útiles Adicionales

Comando	Descripción
`git stash`	Guarda cambios temporalmente sin hacer commit
`git stash pop`	Recupera los cambios guardados con stash
`git clean -f`	Elimina archivos no rastreados
`git reflog`	Muestra un registro de todos los movimientos de HEAD (para recuperar co perdidos)
`git cherry-pick <commit>`	Aplica un commit específico a la rama actual
`git bisect start`	Inicia una búsqueda binaria para encontrar el commit que introdujo un bug

---

💡 Consejos Prácticos

1. **Siempre usa** `git status` **antes de hacer algo importante** para saber en qué estado estás .
2. **Haz commits frecuentes y con mensajes descriptivos**. Es mejor tener muchos commits pequeños que uno enorme .
3. **Antes de hacer** `push`**, haz** `pull` para evitar conflictos con cambios de otros .
4. **Si no estás seguro de un comando**, usa `git <comando> -h` para ver la ayuda rápida (como ya probaste con `git config -h`) .
5. **Las ramas son baratas y rápidas** en Git, así que úsalas sin miedo para experimentar .